

Course: AI Champions Programme

GitHub Copilot Enterprise, Spec-Driven Development, Agentic Software Engineering

NIIT StackRoute Programme Design | Honeywell Software Engineering Context

Program Introduction

The programme follows an incremental engineering journey rather than a collection of AI tool topics. Participants first establish the enterprise context and AI Champion role, understand current engineering pain points and success measures, then build practical GitHub Copilot capability, recap prompt engineering and move into context engineering, learn SDD, practise a complete greenfield SDLC, learn brownfield evolution and best practices, validate SDD-developed features through a proper test strategy, extend agents with MCP, automate PR quality with LLM-as-Judge, enable business and program roles through studio-style workflows in Atlassian Rovo Studio, observe AI workflows with Agent Prism, measure token economics and ROI, and finally demonstrate the complete workflow through the Honeywell Software Engineering Capstone and Adoption Roadmap.

GitHub Copilot Enterprise is used as the primary AI engineering platform. Participants first build a strong working knowledge of Copilot capabilities and workflows, including chat, inline code assistance, agent mode, code explanation, refactoring, test generation, debugging, repository-aware assistance, model/task selection, GitHub.com workflows, terminal/CLI workflows, PR and code-review assistance, and reusable engineering instructions/skills. They then learn two complementary development flavours. In the greenfield flavour, they use SDD to move from problem framing and requirements through specification, architecture, implementation, testing, review, CI/CD and release. In the brownfield flavour, they learn repository understanding, architecture and dependency analysis, change-impact analysis, preservation of existing design principles and patterns, controlled implementation, regression protection and safe evolution. MCP is treated as a key engineering capability for connecting agents to approved tools, repositories, services and enterprise workflows.

A core differentiator of this programme is the direct comparison between prompt-only development and specification-led development, followed by validation through a proper test strategy. Module 3 deliberately recaps practical prompt engineering before moving into context engineering: how to select, structure, minimise and manage repository, design, API, issue, test and specification context. Participants then experience both greenfield and brownfield workflows and measure the impact on implementation cycle time, quality, regression risk, review effort, context usage, token consumption and AI cost. MCP is treated as a major engineering capability for connecting agents to approved tools and enterprise services.

Agent Prism is positioned as the live practice platform for monitoring agent behaviour, traces, failures, token usage, cost leakage, quality signals, and adoption. Participants connect observability to the engineering success metrics rather than treating it as a separate tooling topic. The programme closes with a working brownfield feature-change capstone and a measurable adoption roadmap.

Business Problems This Programme Solves

- Slow feature and application implementation cycles caused by repetitive requirements analysis, architecture, coding, test creation, documentation, CI/CD, and review activities.
- Risk of AI-generated changes disrupting brownfield applications, violating existing architecture or design patterns, breaking contracts, or introducing regressions.
- Weak translation of requirements into precise implementation tasks, leading to inconsistent results when teams rely on prompts alone instead of a structured specification and validation workflow.
- Manual and inconsistent testing and PR review, creating review bottlenecks and making it difficult to validate specification compliance, integration behaviour, and regression safety.
- Excessive token consumption and weak context discipline, resulting in avoidable AI cost and lower-quality outputs.
- Limited visibility into AI usage, quality, failure patterns, token economics, and measurable productivity/ROI from AI-assisted engineering.

Indicative Design and Content Coverage

Pre-requisites

- Participants should have working familiarity with enterprise software engineering environments, SDLC/PDLC practices, Git/GitHub collaboration, and brownfield application development.
- Technical participants should be comfortable with at least one language or stack across Python, .NET Core/C#, ReactJS, Node.js, and relational/database technologies, plus core practices such as branching, pull requests, code review, testing, debugging, and basic design principles/patterns.
- Participants should be prepared to work with an approved GitHub repository and a representative brownfield application. Developers and designers should be able to inspect existing architecture, APIs, dependencies, tests, and coding conventions before making changes.
- Recommended pre-read/self-paced foundation: AI fundamentals bootcamp, intro to GenAI, intro to agents, and responsible AI awareness as listed in the Honeywell training sheet.
- Access should be arranged for GitHub Copilot Enterprise, GitHub Spec Kit or an OpenSpec-equivalent workflow, approved repositories, test projects or a sample brownfield codebase, Docker/Testcontainers, CI/PR pipelines or mock pipelines, and Agent Prism sandbox/demo access.

Course: AI Champions Programme for Software Engineering Teams - NIIT Delivery Design

Total Duration: 24 Hours — 3 Full Days (8 hours/day), delivered as an intensive hands-on programme with a logical progression from enterprise context and Copilot foundations through SDD, greenfield and brownfield software engineering, testing, MCP, PR quality, observability, economics, and the final capstone.

Target Roles

- Common foundation for all: Architects, Developers, Testers, DevOps, SRE, Engineering Leads, Product Owners, Designers, and selected Program/TPO/RTE/Scrum roles
- Technical deep-dive emphasis: Architects, Developers, Testers, DevOps, SRE, Engineering Leads
- Design/collaboration emphasis: Product Owners, Designers, TPOs, RTEs, and Scrum roles

Key Outcomes

- Apply an incremental AI-assisted software engineering lifecycle covering GitHub Copilot capabilities, prompt and context engineering, specification, planning, implementation, testing, PR review, MCP-enabled tool integration, observability, economics and adoption.
- Use GitHub Copilot Enterprise effectively across its major engineering workflows including chat/conversation, inline code assistance, code explanation, refactoring, debugging, test generation, agent mode, repository-aware assistance, model/task selection, terminal/CLI workflows, GitHub.com workflows, code review/PR assistance, and reusable engineering instructions/skills.
- Author structured specifications using a Spec-Driven Development (SDD) workflow — e.g., GitHub Spec Kit or an OpenSpec-equivalent pattern — and use specification, architecture/plan, tasks, and acceptance criteria to drive precise, traceable implementation for greenfield features and applications.
- Enhance brownfield applications safely by identifying architecture, dependencies, extension points, design principles, existing patterns, API contracts, tests, and change impact before modifying code.
- Design and execute a proper test strategy for SDD-developed features, including unit, integration, API/database, contract and E2E testing; use Testcontainers explicitly to validate realistic service and database integrations and protect against regressions.

- Recap practical prompt engineering techniques and then apply context engineering by selecting, structuring, compressing and governing the context supplied to AI. Manage repository context, design notes, APIs, standards, issue history, specifications and tests effectively while keeping token usage and context inflation under control.
- Implement AI-assisted PR quality gates including specification compliance, design/coding standards, security, test evidence, regression risk, and LLM-as-Judge patterns with human escalation where required.
- Build MCP-enabled engineering workflows that connect agents to approved tools, repositories, APIs, databases, test infrastructure, documentation, and enterprise workflows; understand tool boundaries, permissions, approvals, failure handling, and governance. RAG is intentionally excluded from the core programme.
- Use Agent Prism to monitor traces, quality signals, failures, token usage, cost patterns, adoption and ROI, and build an evidence-based 30/60/90-day adoption roadmap.

Detailed Design

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
1. AI Champions Kick-off and Enterprise Context	Programme kick-off, why Honeywell-style software engineering teams need AI champions, shift from isolated AI experimentation to disciplined enterprise adoption, role-based expectations across architects, developers, testers, DevOps, SRE, engineering leads, designers and selected program/product roles, review of the Honeywell training-sheet themes, business and engineering risks of unmanaged AI/ agent usage, introduction to the AI-assisted software engineering lifecycle, and Agent Prism as the monitoring and measurement lens.	2	Workshop: identify current pain points in software engineering delivery, feature implementation, brownfield enhancement, PR cycles, review latency, testing effort and AI usage blind spots. Map current-state versus target-state engineering workflow, identify greenfield and brownfield use cases, define programme success metrics, and establish baseline measures for implementation cycle time, quality/regression risk, PR review time, testing effort, token usage and AI cost.	Slides, role-mapping canvas, current/target-state workflow canvas, success-metrics worksheet, Agent Prism overview demo

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
2. GitHub Copilot Enterprise Features and Engineering Workflows	GitHub Copilot Enterprise capabilities including chat/conversation, inline code assistance, code explanation, refactoring, debugging, test generation, agent mode, repository-aware assistance, model/task selection, GitHub.com workflows, terminal/CLI workflows, PR/code-review assistance, custom/reusable instructions and skills, and safe enterprise usage practices.	2	Hands-on: perform the same engineering tasks using different Copilot capabilities; generate/explain/refactor code, create tests, debug a defect, use agent mode on a repository task, and compare the workflows for speed and quality.	GitHub Copilot Enterprise, IDE, GitHub.com, terminal/CLI, sample repositories
3. Prompt Engineering Recap and Context Engineering	Recap practical prompt engineering for software tasks: task framing, role/instruction, constraints, examples, decomposition, iterative prompting and verification. Move into context engineering: repository context, relevant files/dependencies, architecture/design notes, APIs, standards, issue history, specifications and tests; context selection, layering, compression, task-scoped context, context boundaries, token-aware practices and avoiding context inflation.	2	Hands-on: solve an engineering task with a prompt-only approach, then progressively improve it using structured context. Compare output quality, token usage, iteration count and task completion time.	Copilot, prompt/context templates, repository examples, token usage worksheet
4. Spec-Driven Development with GitHub Spec Kit / OpenSpec	SDD fundamentals, requirements, functional/non-functional constraints, interfaces/contracts, acceptance criteria, specification quality, spec → plan → tasks → implementation, GitHub Spec Kit/ OpenSpec, prompt-only versus spec-driven development, traceability and change control.	2	Hands-on: create a feature specification, generate plan/tasks, implement against the specification, validate acceptance criteria, and compare with a prompt-only baseline.	GitHub Copilot, GitHub Spec Kit / OpenSpec, specification templates, acceptance-criteria checklist

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
5. Software Development – SDD to Complete Software SDLC	Use SDD to build a greenfield application/feature through architecture, project conventions, implementation, test strategy, integration, CI/CD, PR, release readiness and operational considerations. Show how Copilot assists across the complete SDLC rather than only code generation. Repository archaeology, architecture/dependency analysis, change-impact analysis, extension points, existing design principles and patterns, API compatibility, minimal-change strategy, refactoring versus feature implementation, backward compatibility and regression protection.	4	Hands-on: take the specification from Module 4 through implementation, tests, CI/quality checks and PR/release readiness. Validate that the delivered feature matches the specification. Hands-on: analyse an existing application, identify affected components and constraints, create a change plan, implement a feature without unnecessary disruption, verify preservation of existing behaviour, and compare greenfield vs brownfield approaches.	Copilot Enterprise, Spec Kit/OpenSpec, GitHub repo, CI pipeline, test framework, brownfield codebase, architecture/design notes, dependency tools, change-impact checklist
6. Test Strategy, Testcontainers and End-to-End Validation	Test strategy derived from SDD, specification-to-test traceability, unit tests, integration tests, API/database validation, Testcontainers, realistic dependencies, test data, negative scenarios, contract validation, E2E testing, regression testing, coverage and acceptance-criteria validation.	2	Hands-on: derive the test strategy from the SDD feature, create/update unit and integration tests, use Testcontainers explicitly for real service/database dependencies, execute E2E tests, inject a deliberate defect, and prove the strategy catches it.	Docker, Testcontainers, unit/integration test framework, API/database test tools, E2E tooling, CI pipeline

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
7. MCP-Enabled Agentic Engineering Workflows and Reusable Engineering Skills	MCP architecture, servers/tools/resources, connecting agents to approved repositories, APIs, databases, documentation, testing infrastructure and enterprise services; tool selection, permissions, approvals, context/state boundaries, failure handling, security and governance. Package specifications, instructions, skills, prompt packs, test/review heuristics and agent behaviours into reusable cross-project assets with versioning and ownership.	2	Hands-on: build an MCP-enabled workflow such as repository analysis → specification validation → test execution → result analysis or PR evidence collection. Then package one reusable skill such as spec validator, test enhancement assistant or PR reviewer and document its reuse/governance model.	GitHub Copilot/agent environment, MCP server examples, approved tools/APIs, GitHub assets, skill templates, workflow canvas
8. PR Process Automation, Quality Gates and LLM-as-Judge	AI-assisted PR review, specification compliance, coding/design standards, security checks, test evidence, regression risk, maintainability, review heuristics, evidence-based comments, LLM-as-Judge evaluation rubrics, confidence, human escalation and review-cycle reduction.	1.5	Hands-on: configure a PR quality flow, run AI review and LLM-as-Judge checks against a deliberately flawed change, capture evidence, compare review time and quality, and close the loop with human approval.	GitHub repo, PR templates, quality checklist, CI pipeline, LLM-as-Judge rubric

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
9. Studio-Style Workflows for Business and Engineering Collaboration	How non-code or semi-technical users shape agent behaviour through studio-style workflows in Atlassian Rovo Studio: agent authoring for program and product roles, grounding agents on Jira issues, Confluence design and release documentation and other connected engineering sources, translating business intent into agentic execution, triggers and automation rules, actions and connectors, approval flows, human-in-the-loop checkpoints, permission-aware access to project data, and versioning and ownership of published workflows. Covers how business users collaborate with technical owners without bypassing governance, and where the boundary sits between a studio workflow and an engineering agent. Positioned as the enterprise workflow-design layer alongside the Copilot and MCP engineering workflows built earlier.	2	Hands-on: program and product users define an agent-assisted workflow in Atlassian Rovo Studio for program reporting, issue triage, or release coordination. Ground it on a sample Jira project and Confluence space, configure the trigger, actions and approval steps, test against representative project data, review permission and governance implications, and hand it over to technical owners for implementation.	Atlassian Rovo Studio, Jira/ Confluence sandbox tenant, sample project and space, workflow canvas, studio-pattern examples, business-user templates
10. Agent Prism for Monitoring, Observability and Enterprise Control	Why monitoring agentic engineering matters, observability concepts, traces, replay, failure patterns, prompt/output drift, quality/control signals, token usage, cost leak visibility, thresholds and alerts, adoption tracking, operational review, and connecting AI telemetry to engineering outcomes.	1.5	Hands-on: ingest or simulate engineering-agent run data in Agent Prism, inspect traces and failures, identify token/cost leakage, compare successful and poor-quality runs, and relate findings to implementation, review and testing KPIs.	Agent Prism sandbox/demo, sample run telemetry, monitoring scorecards

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
11. ROI, Token Economics and Deployment Governance	Engineering productivity versus production impact, cost per accepted production-quality output, token governance, saved implementation/review/testing effort, quality and regression signals, business-value measurement, when an AI workflow should be scaled/tuned/restricted/retired, governance and leadership reporting.	1	Hands-on: calculate before/after metrics for the engineering workflow, estimate AI cost per accepted production-quality output, build an ROI model using saved effort and reduced review/testing time, and define governance actions.	ROI worksheet, token economics template, Agent Prism metrics, KPI dashboard
12. Honeywell Software Engineering Capstone and Adoption Roadmap	Teams integrate the complete engineering journey into one realistic adoption plan: GitHub Copilot Enterprise workflow, reusable skill/spec pack, greenfield SDD or brownfield enhancement approach, proper test strategy with Testcontainers/E2E, MCP-enabled workflow, PR quality automation, LLM-as-Judge, context/token discipline, and Agent Prism monitoring model. Final review includes business case, governance controls, and enterprise rollout roadmap.	2	Capstone: present one software engineering use case with solution blueprint, specification/plan, implementation approach, test strategy and evidence, MCP workflow, PR/LLM-Judge evidence, Agent Prism monitoring design, ROI model, and 90-day adoption roadmap.	Greenfield starter or brownfield application, GitHub repo/PR pipeline, Spec Kit/OpenSpec, Testcontainers, MCP tools, Agent Prism, capstone canvas, evaluation rubric

Coverage Alignment to the Uploaded Honeywell Training Sheet

- The programme is deliberately structured as a 12-module progression so each concept builds on the previous one: enterprise context and AI Champion role → GitHub Copilot capabilities → prompt engineering recap and context engineering → SDD → greenfield SDLC → brownfield engineering → test strategy/ Testcontainers/E2E → MCP and reusable skills → PR quality/LLM-as-Judge → Agent Prism → ROI/token economics/governance → Honeywell Software Engineering Capstone and Adoption Roadmap.
- Discovery and Launch themes are reflected through design thinking, requirements framing, greenfield application planning, brownfield repository understanding, context engineering, structured specifications, use-case identification, and translating requirements into implementation-ready work.
- Development and Test themes are strengthened through GitHub Copilot agent mode, SDD, complete SDLC execution, repository/context management, AI-assisted implementation, design-principle compliance, Testcontainers, integration/E2E testing, PR automation, code review, LLM-as-Judge, reusable skills, MCP/ tool integration, and AI usage cost optimisation.
- COTS AI tool themes are retained through MCP server and tool-integration patterns. RAG is intentionally excluded so the available time can be invested in deeper SDD, MCP, testing, brownfield engineering, and complete SDLC practice.
- Agent Prism remains the practical enterprise monitoring and observability layer so teams can connect AI-assisted engineering activity with quality, token usage, cost, adoption, and ROI.

Learning Pedagogy

- Instructor-led sessions use two realistic software engineering journeys: build a greenfield application feature using SDD and manage a change in an existing brownfield application without disrupting established behaviour. GitHub Copilot capabilities are introduced first so participants have a practical tool foundation before applying the engineering methods.
- Guided labs follow a complete engineering lifecycle: understand/frame the problem, use Copilot effectively, establish context, specify, plan, implement, test, review, integrate tools through MCP, observe AI behaviour, measure economics, and demonstrate adoption. The brownfield path adds repository archaeology, impact analysis and regression protection.
- Participants recap prompt engineering and then compare prompt-only implementation with structured context and SDD workflows, evaluating differences in implementation time, output quality, context usage, testing effort, review effort, specification compliance, token usage and cost.
- Design-thinking activities help participants frame the problem, identify constraints, understand existing architecture, and define acceptance criteria before asking AI to implement a solution.
- Testing and PR quality clinics connect AI productivity directly to confidence: test strategy from the specification, unit/integration testing, explicit Testcontainers-based validation, E2E testing, regression checks, specification compliance, and LLM-as-Judge review.
- MCP is taught as a practical mechanism for extending AI-assisted engineering with governed tools and enterprise context. Participants design and use MCP-enabled workflows for repository operations, testing, APIs, documentation, or other approved engineering tools.
- Agent Prism labs connect traces, token usage, cost, quality and failure patterns to the programme success metrics. The ROI/governance module then converts these observations into measurable business outcomes before the final capstone.

Free Value-Adds / Client Handholding

- 30-day catch-up review session at no additional cost to review what participants have applied, what has been deployed, and where adoption is blocked.
- One complimentary masterclass after the 30-day review focused on advanced lessons learned, common implementation mistakes, SDD maturity, testing strategies, and next-step AI engineering patterns.
- Post-training assessment framework to review feature implementation cycle time, PR review time, regression/defect signals, reusable skill/spec adoption, token spend discipline, and early ROI signals from deployed or piloted workflows.
- Short gap-closure recommendations based on participant progress, leadership expectations, and identified blockers in governance, tooling, brownfield adoption, testing, or workflow design.

Delivery Recommendations

- Recommended audience: Honeywell software engineering teams with role-based participation from architects, developers, testers, DevOps, SRE, engineering leads, designers, and selected product/program roles.
- Preferred delivery model: blended delivery aligned to the uploaded sheet - overview sessions online, use-case identification/design-thinking workshops interactive, and hands-on brownfield engineering working sessions in person for shortlisted use cases.
- Primary tool/platform: GitHub Copilot Enterprise for core AI-assisted development workflows, GitHub Spec Kit / OpenSpec for SDD, Docker and Testcontainers for integrated testing, GitHub PR/CI workflows for quality gates, MCP servers/tools for governed engineering integrations, and Agent Prism for monitoring, observability, token economics and ROI.
- Recommended batch design: common foundation followed by two targeted engineering journeys — greenfield SDD/complete SDLC and brownfield enhancement/safe evolution — with Copilot capabilities and prompt/context engineering established first, and MCP integrated after the core development workflow is understood. RAG is intentionally excluded from the core programme.
- Outcome expectation: every batch should leave with a working greenfield SDD workflow, a working brownfield feature-change approach, a documented test strategy with Testcontainers/E2E evidence, reusable engineering assets, MCP-enabled workflow patterns, PR quality automation, Agent Prism observability evidence, a token/cost and ROI model, and a completed Honeywell Software Engineering Capstone and Adoption Roadmap.
- Client enablement needed: approved GitHub access, a representative greenfield application starter and brownfield full-stack repository, architecture/design documentation, coding standards, existing tests, Docker/ Testcontainers support, approved MCP servers/tools, PR/CI pipelines or mock pipelines, reusable standards/ checklists, and Agent Prism practice environment or demo dataset.